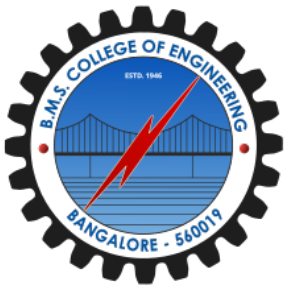
B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU)

Bull Temple Road, Basavangudi, Bengaluru – 560019



Bio-inspired Algorithms AAT report on

Concrete Compressive Strength Prediction Using Differential Evolution–Optimized Support Vector Regression

in partial fulfillment of the requirements for the award of degree

BACHELOR OF ENGINEERING

in

Computer Science and Engineering (Data Science)

by

R V Abhishek (1BM23CD047)

Sushanth S (1BM23CD063)

Ullas N (1BM23CD067)

Vedantha V V (1BM23CD069)

Under the Guidance of

Prof. Manoj Kumar

Assistant Professor

Department of CSE (Data Science)

B.M.S. College of Engineering

AY 2026–27

B.M.S. COLLEGE OF ENGINEERING

(Autonomous College under VTU)

Bull Temple Road, Basavangudi, Bengaluru – 560019

Department of Computer Science and Engineering (Data Science)

CERTIFICATE

This is to certify that the project entitled "**Concrete Compressive Strength Prediction Using Differential Evolution–Optimized Support Vector Regression**" is the work submitted by the following students for **Bio-inspired Algorithms (23DS7BSBIO)** during academic year 2026–27, under the guidance of Prof. Manoj Kumar, Assistant Professor, Department of Computer Science and Engineering (Data Science), B.M.S. College of Engineering. The report is submitted in partial fulfillment of the academic requirements of the course.

Candidate details

No.	Student name	USN	Signature
1	R V Abhishek	1BM23CD047	
2	Sushanth S	1BM23CD063	
3	Ullas N	1BM23CD067	
4	Vedantha V V	1BM23CD069	

Signature of the Guide
Prof. Manoj Kumar
Assistant Professor

Signature of the HOD
Dr. Shyamala G
Head of Department

Examiners

1. _____ _____

2. _____ _____

Acknowledgement

We thank Prof. Manoj Kumar, Assistant Professor, Department of Computer Science and Engineering (Data Science), B.M.S. College of Engineering, for guiding this Bio-inspired Algorithms Alternative Assessment project. We thank Dr. Shyamala G, Head of the Department, and the department faculty for the academic environment and resources supporting this work.

We acknowledge I-Cheng Yeh and the UCI Machine Learning Repository for making the Concrete Compressive Strength dataset available. We also acknowledge the authors and maintainers of the scientific Python libraries used in the implementation. Their datasets, publications and software are cited in the report.

The project was completed by R V Abhishek, Sushanth S, Ullas N and Vedantha V V. The division of work covered data preparation and baselines, Differential Evolution implementation, final evaluation, and report and presentation preparation.

Contents

Acknowledgement iii

List of Abbreviations ix

Abstract x

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Research Objectives . . . 1
- 1.4 Contributions . . . 2
- 1.5 Paper Organization . . . 2

2 Literature Review 3
- 2.1 Concrete Strength Prediction Techniques . . . 3
- 2.2 Machine Learning for Concrete Strength Prediction . . . 3
- 2.3 Hyperparameter Optimization Methods . . . 3
- 2.4 Evolutionary Algorithms in Engineering Applications . . . 4
- 2.5 SVR Optimization with Evolutionary Methods . . . 4
- 2.6 Concrete Dataset Studies . . . 4
- 2.7 Research Gap . . . 4

3 Theoretical Background 5
- 3.1 Regression and Generalization . . . 5
- 3.2 Support Vector Regression . . . 5
- 3.3 Radial Basis Function Kernel . . . 5
- 3.4 Feature Scaling . . . 5
- 3.5 Differential Evolution Fundamentals . . . 6
- 3.6 Evaluation Metrics . . . 6

4 Nature-Inspired Algorithms: Detailed Study 7
- 4.1 Overview of Nature-Inspired Optimization . . . 7
- 4.2 Genetic Algorithm: In-Depth Analysis . . . 7
- 4.3 Particle Swarm Optimization . . . 7
- 4.4 Ant Colony Optimization . . . 7
- 4.5 Simulated Annealing . . . 7
- 4.6 Differential Evolution . . . 8

4.7 Why Differential Evolution for SVR Optimization? . . . 8
4.8 Hybrid Approaches . . . 8

5 Methodology and Implementation 9
5.1 System Architecture . . . 9
5.2 Data Collection . . . 9
5.3 Data Preprocessing . . . 9
5.4 Feature Engineering . . . 10
5.5 SVR Model Architecture . . . 10
5.6 Differential Evolution Implementation . . . 10
5.7 Training Procedure . . . 11
5.8 Implementation Details . . . 11

6 Experimental Setup and Test Functions 12
6.1 Experimental Configuration . . . 12
6.2 Baseline Models . . . 12
6.3 Test Functions and Benchmarks . . . 12
6.4 Cross-Validation Strategy . . . 13
6.5 Sensitivity Analysis . . . 13
6.6 Statistical Significance Testing . . . 13
6.7 Computational Efficiency Metrics . . . 14

7 Results and Discussion 15
7.1 DE Convergence Analysis . . . 15
7.2 Optimal Hyperparameter Configuration . . . 15
7.3 Prediction Performance . . . 15
7.4 Temporal Performance Analysis . . . 16
7.5 Comparison with Baseline Models . . . 17
7.6 Hyperparameter Importance Analysis . . . 17
7.7 Feature Importance Results . . . 17
7.8 Error Analysis . . . 17
7.9 Computational Performance . . . 18
7.10 Robustness Analysis . . . 18
7.11 Discussion . . . 19

8 Comparative Performance Analysis 20
8.1 Accuracy and Search Budget . . . 20
8.2 Advantages and Limitations . . . 20

9 Applications and Case Studies 21

9.1 Applications 21
9.2 Case Study: UCI Concrete Compressive Strength 21

10 Conclusion **22**
10.1 Key Findings 22
10.2 Key Learnings 22

11 Future Work **23**
11.1 Methodological Enhancements 23
11.2 Data and Validation 23
11.3 Sustainable Development Goals 23

References **24**

GitHub Repo Link **25**

List of Figures

7.1 Development validation convergence at equal candidate budgets. 15
7.2 Holdout RMSE: default SVR and five-seed tuned-model aggregates. Error bars describe optimizer-seed SD. 16
7.3 Predictions and residuals for the validation-selected demonstration, DE seed 11. 17
7.4 Three-fold development diagnostic with nested whole-group samples and fixed demonstration parameters. 19

List of Tables

5.1 Dataset variables and units. 9

6.1 Final experimental configuration. 12
6.2 Saved synthetic benchmark best objective values (lower is better). 13

7.1 Holdout results; tuned values are five-seed mean ± sample SD. 16
7.2 All ten predefined validation-selected runs, evaluated on the same holdout. . . 18

List of Abbreviations

ACO	Ant Colony Optimization
AAT	Alternative Assessment Tool
CV	Cross-validation
DE	Differential Evolution
GA	Genetic Algorithm
MAE	Mean Absolute Error
MPa	Megapascal
PSO	Particle Swarm Optimization
RBF	Radial Basis Function
RMSE	Root Mean Square Error
RS	Random Search
SD	Sample standard deviation
SVR	Support Vector Regression
UCI	University of California, Irvine
USN	University Seat Number

Abstract

Predicting concrete compressive strength from mixture composition and curing age is a nonlinear regression problem. Support Vector Regression (SVR) can model these relationships, but its performance depends on the regularization parameter, radial basis function kernel width and epsilon-insensitive loss threshold. This project evaluates whether Differential Evolution (DE) provides a useful automated method for selecting these hyperparameters under a fixed computational budget.

The study uses the UCI Concrete Compressive Strength dataset with 1,030 observations, eight numerical inputs and compressive strength as the target in megapascals. Identical input tuples are assigned to common groups before splitting. The fixed partition contains 826 development observations and 204 holdout observations. A StandardScaler–SVR pipeline is fitted separately inside each of five group-disjoint development folds, avoiding leakage from preprocessing or repeated input combinations. DE/rand/1/bin is implemented explicitly with differential mutation, binomial crossover, forced mutant-coordinate inheritance, clipped bounds and deferred greedy selection.

Default RBF-SVR is compared with Random Search and DE using the same three-dimensional logarithmic search space. Each tuned method receives 300 candidate evaluations per seed, including initialization, for five predefined optimizer seeds. Candidate selection uses mean development cross-validation RMSE. The original baseline and Random Search holdout evaluations preceded final DE evaluation; this chronology is retained rather than describing the entire study as prospectively blinded.

Default SVR obtains holdout RMSE of 9.9069 MPa, MAE of 7.8854 MPa and an R^2 of 0.5755. Random Search achieves mean RMSE of 6.2328 ± 0.1194 MPa, while DE achieves 6.2250 ± 0.1104 MPa across optimizer seeds. DE reduces mean holdout RMSE by approximately 37.2% relative to default SVR. Its difference from Random Search is only about 0.008 MPa, so the results support the value of hyperparameter tuning without establishing significant superiority of DE. Convergence, prediction-residual and development learning-curve diagnostics accompany the comparison. The contribution is a reproducible, budget-controlled experimental pipeline and an auditable implementation of DE; broader deployment and statistical generalization require further validation.

Keywords: concrete compressive strength; Differential Evolution; Support Vector Regression; hyperparameter optimization; group cross-validation; reproducibility.

Chapter 1

Introduction

1.1 Background and Motivation

Concrete compressive strength depends on the interaction of constituent materials and curing age. The relationship is not adequately described by treating every ingredient as an independent linear contribution. The UCI dataset provides measurements of cement, blast furnace slag, fly ash, water, superplasticizer, coarse aggregate, fine aggregate and age, together with measured compressive strength [1], [2]. A predictive model can support the study of these relationships within the range represented by the data.

This project treats strength prediction as supervised numerical regression. It does not replace physical testing or certify a mixture for construction. The immediate goal is to compare ways of configuring one established regression model under a reproducible protocol. Keeping the model and data fixed makes the effect of hyperparameter selection easier to interpret.

RBF-SVR is a suitable compact model for this exercise because it captures nonlinear relationships through a kernel and exposes a small set of influential hyperparameters [3]. Default values provide a reproducible starting point, but they need not match the numerical scale or geometry of this dataset. Automated search therefore forms the optimization component of the project.

1.2 Problem Statement

The task is to predict compressive strength y in MPa from an eight-dimensional vector $\mathbf{x}$ of ingredients and age. The optimization task is to select C, γ and ϵ for RBF-SVR by minimizing mean development-fold RMSE. The comparison must distinguish improvement from tuning itself from improvement specifically attributable to the DE search strategy.

A useful result requires identical evaluation folds, fold-local preprocessing, an equal candidate budget and a comparator that explores the same parameter space. Otherwise differences in data leakage, search range or computational effort can be mistaken for an algorithm advantage.

1.3 Research Objectives

1. Validate the dataset schema, units, missingness and repeated input combinations.
2. Establish a reproducible default-SVR baseline and a comparable Random Search baseline.

3. Implement DE mutation, crossover, selection and bound handling explicitly.
4. Compare validation and holdout RMSE, MAE and R^2 using five predefined optimizer seeds.
5. Preserve candidate histories, selected configurations, predictions, diagnostics and provenance for independent inspection.

1.4 Contributions

The work contributes an auditable DE/rand/1/bin implementation integrated with a shared CV evaluator, a group-disjoint split contract, and a complete frozen evidence package. It also provides a corrected development learning curve based on nested whole-group samples, including both training and validation error. These are engineering and experimental contributions; this report does not claim a new optimization algorithm or a universally superior prediction method.

1.5 Paper Organization

The literature review is followed by theoretical background, a detailed study of nature-inspired algorithms, methodology and implementation, experimental setup, and results and discussion. Comparative performance analysis and applications place the measurements in context. The report closes with the conclusion, future work, references and repository link.

Chapter 2

Literature Review

2.1 Concrete Strength Prediction Techniques

Yeh's 1998 work studied strength modeling for high-performance concrete using artificial neural networks [2]. It provides the domain background for the dataset used here. The dataset is valuable for testing nonlinear regression because inputs include both mixture composition and curing age. Published results from another paper cannot be transferred directly to this study without reproducing its partition and evaluation procedure.

2.2 Machine Learning for Concrete Strength Prediction

A supervised model learns an association between measured inputs and target strength. Neural networks and kernel methods offer different representations of nonlinear relationships. This project deliberately evaluates one RBF-SVR model rather than expanding the comparison to many learners. A small scope allows stronger control of preprocessing, data isolation and optimizer budgets.

The dataset is cross-sectional tabular data. Age is a feature describing curing duration; it does not make the rows an ordered financial time series. Temporal forecasting procedures are therefore not appropriate for the fixed tabular evaluation used here.

2.3 Hyperparameter Optimization Methods

Hyperparameters are selected outside the model's fitting procedure. Grid search enumerates a fixed lattice; Random Search draws candidates from a specified distribution. Bergstra and Bengio establish Random Search as an effective hyperparameter optimization baseline, motivating its inclusion as a comparator in our DE–SVR experiments [4]. Their paper compares Random Search with grid and manual search; it does not establish an advantage or disadvantage relative to DE. For this experiment, it provides an especially direct control: its parameter bounds, candidate count, folds and seeds match those used by DE.

Equal candidate counts control the number of objective calls, but do not guarantee equal wall-clock duration. Some SVR configurations take longer to fit. Runtime is therefore recorded separately and is not included in the selection objective.

2.4 Evolutionary Algorithms in Engineering Applications

Evolutionary methods maintain populations or adapt candidates through stochastic variation and selection. Their appeal is the ability to search without differentiating the objective through the complete model-training pipeline. This is relevant when fitness is a cross-validation score obtained from repeated SVR fitting.

Storn and Price introduced DE as a method for continuous optimization using scaled differences between population vectors [5]. DE is used here because the three searched coordinates are continuous logarithmic parameters. This choice does not establish that DE is preferable for every engineering problem.

2.5 SVR Optimization with Evolutionary Methods

Combining a search method with SVR creates an outer optimization loop around a fixed inner estimator. The candidate determines the estimator parameters; the evaluator fits a fresh pipeline in each development fold and returns mean RMSE. The search never receives holdout predictions as fitness. This separation is more consequential for validity than the label of the outer optimizer.

The present implementation fixes mutation weight, crossover rate and population size before final experiments. No adaptive DE extension, ensemble, feature selector or additional learner is introduced. This keeps attribution of observed changes straightforward.

2.6 Concrete Dataset Studies

The UCI repository describes 1,030 instances, eight features and no missing attribute values [1]. Ingredient quantities are measured in kilograms per cubic metre, age in days, and strength in MPa. Repeated complete input tuples occur in the local data. Random row splitting could place identical inputs on both sides of an evaluation boundary, so this project groups those tuples before splitting.

2.7 Research Gap

The practical question is whether explicitly implemented DE improves upon an equally budgeted random search for this particular SVR task. The study addresses experimental comparability and reproducibility rather than claiming an unexplored application area. Its limited dataset and fixed partition permit a controlled course project but restrict general conclusions about algorithm rankings.

Chapter 3

Theoretical Background

3.1 Regression and Generalization

A regression model estimates $\hat{y} = f(\mathbf{x})$ from paired measurements. Training error measures fit to observations used by the estimator; validation error guides hyperparameter selection; holdout error evaluates fixed configurations on a reserved partition. These quantities answer different questions and should not be combined into one score.

3.2 Support Vector Regression

Epsilon-SVR balances a regularized function with deviations outside an epsilon-insensitive tube. A standard primal form is

$$\min_{\mathbf{w},b,\xi,\xi^*} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}(\xi_i + \xi_i^*) \tag{3.1}$$

subject to $y_i - \langle \mathbf{w}, \phi(\mathbf{x}_i)\rangle - b \leq \epsilon + \xi_i$, $\langle \mathbf{w}, \phi(\mathbf{x}_i)\rangle + b - y_i \leq \epsilon + \xi_i^*$ and nonnegative slack variables. Larger C places a larger penalty on violations. Epsilon controls which residuals are ignored by the loss. These controls interact with the kernel and feature scale [3], [6].

3.3 Radial Basis Function Kernel

The RBF kernel is

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2). \tag{3.2}$$

Gamma controls the spatial scale of similarity. A large value can produce a highly local response; a small value produces a broader response. Neither extreme is assumed to be best. Cross-validation is used to select the combination of gamma, regularization and loss tolerance.

3.4 Feature Scaling

The scaler transforms a coordinate using statistics estimated on the current fitting subset:

$$z_j = \frac{x_j - \mu_j}{\sigma_j}. \tag{3.3}$$

Ingredients and age have different numerical scales. Standardization prevents an input's raw units alone from dominating Euclidean distances. In each fold the scaler learns μ_j and σ_j from

the training rows only; the validation rows are transformed using those same values. The target remains in MPa.

3.5 Differential Evolution Fundamentals

Each candidate encodes $\mathbf{z} = (\log_{10} C, \log_{10} \gamma, \log_{10} \epsilon)$. Parameters are decoded with $\theta_j = 10^{z_j}$. Uniform search in log coordinates covers multiplicative scales without allocating nearly all samples to large linear values.

The scalar fitness is

$$J(\mathbf{z}) = \frac{1}{5} \sum_{k=1}^{5} \mathrm{RMSE}_k(\mathbf{z}). \tag{3.4}$$

Lower fitness is better. This objective contains no runtime term or holdout score.

3.6 Evaluation Metrics

For actual values y_i, predictions $\hat{y}_i$ and mean $\bar{y}$, the metrics are

$$\mathrm{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}, \tag{3.5}$$

$$\mathrm{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|, \tag{3.6}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \tag{3.7}$$

RMSE emphasizes large errors; MAE reports mean absolute deviation in MPa; R^2 compares squared error with prediction of the holdout mean. Across seeds, the report uses arithmetic means and sample standard deviations with denominator $m - 1$. These seed SDs describe search variability on one data partition.

Chapter 4

Nature-Inspired Algorithms: Detailed Study

4.1 Overview of Nature-Inspired Optimization

Nature-inspired algorithms use mechanisms motivated by evolution, collective behavior or physical processes. Common families include evolutionary methods such as GA and DE, swarm methods such as PSO and ACO, and physical-process methods such as simulated annealing. The classification describes motivation; it does not provide a ranking of performance.

4.2 Genetic Algorithm: In-Depth Analysis

A GA maintains a population of encoded candidates, selects parents, applies crossover and mutation, and selects survivors. Its representation may encode discrete or continuous choices. Selection favors better fitness while random variation permits new combinations. Design choices include representation, parent selection, variation rates and survivor policy [7].

GA provides a useful conceptual comparison with DE. It is not an implemented comparator in this project. DE differs from a conventional GA by constructing continuous perturbations from differences among population members rather than requiring the same chromosome operators used in conventional genetic representations.

4.3 Particle Swarm Optimization

PSO updates candidate positions using velocity influenced by individual and social information [8]. It provides an alternative derivative-free strategy for continuous parameters. This study does not implement PSO, tune its controls or report its performance. Such a comparison would require the same folds and budget discipline used for the existing methods.

4.4 Ant Colony Optimization

ACO models constructive search with probabilistic choices influenced by pheromone-like information [9]. It is well known for combinatorial settings. Adapting it to continuous SVR hyperparameters requires additional representation choices, so it is outside this project's fixed scope.

4.5 Simulated Annealing

Simulated annealing probabilistically accepts some worsening moves according to a temperature schedule, allowing exploration beyond strictly greedy local moves [10]. It is included

as a physical-process-inspired method. Its schedule and proposal distribution would need an independently specified protocol before comparison.

4.6 Differential Evolution

For target vector $\mathbf{x}_i$, choose distinct indices r_1, r_2, r_3 that exclude i. Differential mutation computes

$$\mathbf{v}_i = \mathbf{x}_{r_1} + F(\mathbf{x}_{r_2} - \mathbf{x}_{r_3}). \tag{4.1}$$

Coordinates are clipped to their bounds. Binomial crossover creates a trial with

$$u_{ij} = \begin{cases} v_{ij}, & r_j < CR \text{ or } j = j_{\text{rand}}, \\ x_{ij}, & \text{otherwise.} \end{cases} \tag{4.2}$$

The forced index ensures at least one coordinate comes from the mutant. Greedy selection accepts $\mathbf{u}_i$ when its fitness is no worse than that of $\mathbf{x}_i$. The implementation uses a frozen source population within a generation and commits replacements after that generation, matching deferred updating [5].

4.7 Why Differential Evolution for SVR Optimization?

The search has three real-valued coordinates and a fitness obtained from model fitting. DE supports this continuous representation without requiring derivatives. Its operators can be implemented and tested directly within the course scope. The choice is justified by fit to the task and implementation transparency, not by an assumed accuracy advantage over random search.

4.8 Hybrid Approaches

DE–SVR is a hybrid of an outer optimizer and an inner supervised estimator. The search finds parameter settings; SVR learns the prediction function. More elaborate combinations, including feature selection, adaptive controls and ensembles, are possible future experiments. They are not part of the reported evidence.

Chapter 5

Methodology and Implementation

5.1 System Architecture

The pipeline consists of raw-data validation, deterministic group assignment, split-manifest creation, development-only search, fixed-candidate refitting and holdout evaluation. Results are written as structured evidence rather than transcribed from console output. The same evaluator is used by both tuned methods, so preprocessing and fitness definitions remain consistent.

The main modules are `data.py` for ingestion and grouping, `evaluation.py` for fitting and scoring, `search.py` for Random Search, `de.py` for DE operators, and `stage3.py` for finalization and diagnostics. Verification logic audits saved records separately from optimization.

5.2 Data Collection

The dataset is the original UCI Concrete Compressive Strength spreadsheet and its readme [1]. Raw files and the downloaded archive are retained unchanged. The data are not scraped from a live service and are not augmented with another dataset. Source attribution includes both the UCI dataset DOI and Yeh's original publication.

Table 5.1: Dataset variables and units.

Variable	Unit and role
Cement; blast furnace slag; fly ash	kg/m^3, predictors
Water; superplasticizer	kg/m^3, predictors
Coarse aggregate; fine aggregate	kg/m^3, predictors
Age	Days, predictor
Compressive strength	MPa, target

5.3 Data Preprocessing

The spreadsheet headers are mapped explicitly to canonical columns. The expected schema contains eight predictors and one target. Missing values are checked before splitting; no imputation is needed. All 1,030 rows are retained. There are 996 unique complete input tuples, so 34 rows repeat an existing eight-feature combination.

Rows with the same complete input tuple share a group. GroupShuffleSplit with seed 42 reserves 200 groups containing 204 rows for holdout; 796 groups containing 826 rows remain for development. Five GroupKFold splits are fixed inside development data. This isolates

identical input tuples across fitting and evaluation boundaries while retaining their observed target values.

5.4 Feature Engineering

The model uses the eight original numerical inputs. It does not introduce technical indicators, polynomial interactions, feature selection or target transformations. Standardization is the only learned feature transformation. Keeping the feature representation fixed helps isolate the effect of hyperparameter optimization.

5.5 SVR Model Architecture

Each candidate instantiates a StandardScaler followed by an RBF SVR. The default comparator uses $C = 1$, $\epsilon = 0.1$ and gamma set to `scale`. Tuned candidates use decoded numerical values for all three parameters [3]. The scaler is refitted in every training fold and again on all development data for the final selected-model refit.

5.6 Differential Evolution Implementation

Population size is 15, mutation weight $F = 0.8$, crossover rate $CR = 0.9$ and the initialization-inclusive budget is 300. This permits one initial population and 19 complete trial generations. Every attempted fitness evaluation counts, including repeated positions. There is no hidden polishing stage or candidate cache that changes accounting.

Algorithm 5.1 Budget-controlled DE/rand/1/bin

Require: Bounds, population $N = 15$, budget $B = 300$, seed
Initialize N candidates uniformly in log coordinates
Evaluate all candidates using five-fold development RMSE
for $g = 1$ to 19 **do**
    Freeze current population and fitness values
    for each target i **do**
        Select three distinct donors excluding i
        Mutate with $F = 0.8$ and clip to bounds
        Cross over with $CR = 0.9$, forcing one mutant coordinate
        Evaluate trial; retain it if no worse than target
    end for
    Commit the next population
end for
return Candidate with lowest observed development RMSE

5.7 Training Procedure

For each seed, the optimizer receives the same saved five-fold development splits. The selected candidate minimizes mean fold RMSE. Each selected configuration is then refitted on all 826 development rows and evaluated on the same 204 holdout rows. Five seeds, 11, 22, 33, 44 and 55, are retained for each method.

The demonstration model is DE seed 11 because it has the lowest validation score under the predefined tie rule. Its holdout score is not used to choose it. Aggregate results include every predefined seed, rather than only this demonstration.

5.8 Implementation Details

The independent repository uses Python and `uv` with a committed lockfile. NumPy implements the explicit search operators; scikit-learn supplies scaling, SVR, splitting and metrics; pandas handles tabular data. Tests cover donor uniqueness, exclusion of the target, clipping, forced crossover, greedy selection, deterministic seeds and budget accounting.

Original final experiments were run on Windows with Python 3.13.1, NumPy 2.4.6, scikit-learn 1.6.1 and SciPy 1.15.3. Locked macOS verification used Python 3.12, NumPy 2.5.4, scikit-learn 1.9.1 and SciPy 1.18.1. These environments are distinguished in provenance; original results are preserved.

Chapter 6

Experimental Setup and Test Functions

6.1 Experimental Configuration

The main experiment fixes the dataset, holdout, development folds, model family, search bounds and candidate budget before final DE evaluation. Table 6.1 summarizes the controls. The two tuned methods receive $2 \times 5 \times 300 \times 5 = 15{,}000$ tuning-fold fits in total, excluding baselines, final refits and diagnostics.

Table 6.1: Final experimental configuration.

Setting	Value
Development / holdout rows	826 / 204
Development / holdout groups	796 / 200
Development fitness	Mean RMSE, five group-disjoint folds
Optimizer seeds	11, 22, 33, 44, 55
Candidate budget	300 per method per seed, including initialization
DE population / generations	15 / 19 after initialization
Mutation / crossover	0.8 / 0.9
Search coordinates	$\log_{10} C \in [-2, 4]$, $\log_{10} \gamma \in [-5, 1]$; $\log_{10} \epsilon \in [-3, 1]$
Selection	Lowest development CV RMSE

6.2 Baseline Models

Default SVR measures the performance of an untuned configuration. Random Search samples uniformly from the same logarithmic bounds used by DE and receives the same 300 calls per seed. It is an active tuning baseline, not a deliberately weak comparator [4]. The difference between default and tuned results measures the value of tuning; the difference between the two tuned methods addresses search strategy.

6.3 Test Functions and Benchmarks

Before relying on a model-fitting objective, the optimizer was checked on synthetic functions with known minima. Sphere is

$$f(\mathbf{x}) = \sum_j x_j^2, \tag{6.1}$$

and Rastrigin is

$$f(\mathbf{x}) = 10d + \sum_j \left[x_j^2 - 10\cos(2\pi x_j)\right]. \tag{6.2}$$

Both have global minimum zero at the origin. Sphere tests progress on a smooth landscape; Rastrigin adds many local minima. These checks validate behavior on simple objectives and do not predict concrete-strength generalization. The synthetic results remain separate from

Table 6.2: Saved synthetic benchmark best objective values (lower is better).

Function	Seed	DE	Random Search
Sphere	11	0.0717	2.2232
Sphere	22	0.0342	0.5466
Sphere	33	0.0675	0.3155
Sphere	44	0.0538	1.5079
Sphere	55	0.1117	0.6162
Rastrigin	11	11.3277	11.3395
Rastrigin	22	7.6343	5.0592
Rastrigin	33	9.1857	15.1526
Rastrigin	44	4.0336	11.3815
Rastrigin	55	6.2468	9.2295

final regression metrics. In particular, a better Sphere value cannot be presented as a strength-prediction accuracy result.

6.4 Cross-Validation Strategy

Identical eight-feature tuples define groups for both outer splitting and inner folds. The held-out groups never appear in development folds. Within a fold, the scaler is fitted only to training rows. The fixed split manifest retains row IDs and group IDs so the isolation contract can be checked independently.

The historical baseline and Random Search holdout evaluations occurred in Stage 1. Final DE candidates were selected using development validation, but the whole project cannot be described as a prospectively untouched holdout study. This limitation is retained in the interpretation.

6.5 Sensitivity Analysis

The study varies optimizer seeds under fixed controls. It does not perform a factorial sensitivity study of population size, mutation weight or crossover rate. Likewise, it does not establish causal importance of individual hyperparameters. Learning-curve diagnostics examine the effect of increasing whole-group training subsets while using fixed demonstration parameters.

6.6 Statistical Significance Testing

No confirmatory significance test is claimed. Five optimizer seeds share the same data partition and are not five independent datasets. Their SDs describe stochastic search variability. Given

the small DE–Random Search difference, an assertion of statistically significant superiority would require additional design and evidence.

6.7 Computational Efficiency Metrics

Each history retains call counts and elapsed times, while final records retain refit times and environment provenance. Equal objective budgets make evaluation effort comparable; wall-clock times remain dependent on selected parameters and execution environment. Timing observations are not mixed across Windows experiments and macOS verification to manufacture a speed comparison.

Chapter 7

Results and Discussion

7.1 DE Convergence Analysis

Figure 7.1 shows the saved best-so-far development RMSE histories for both search methods. The horizontal axis counts objective evaluations, including initialization. Curves indicate optimization progress on the fixed development folds, not test accuracy. A lower validation endpoint does not guarantee a lower holdout error on a particular seed.

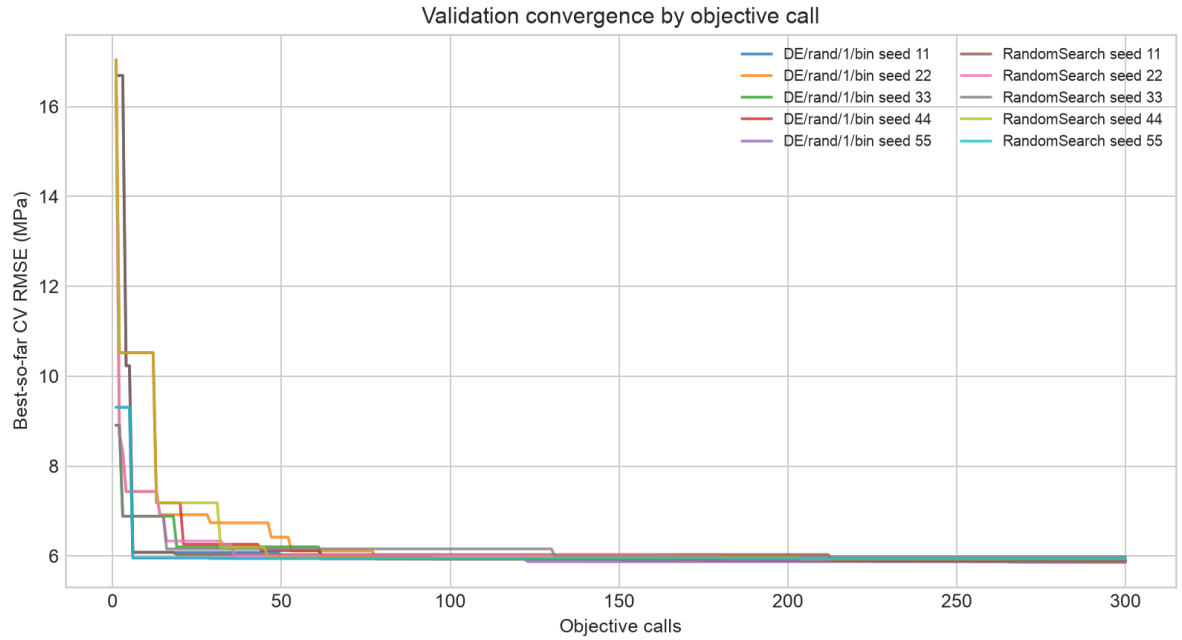


Figure 7.1: Development validation convergence at equal candidate budgets.

7.2 Optimal Hyperparameter Configuration

The best observed validation-selected demonstration is DE seed 11, with CV RMSE 5.8601 MPa. The decoded values are $C = 1462.1749$, $\gamma = 0.0573784$ and $\epsilon = 1.3447436$. "Optimal" refers to the best observed candidate in this finite search; no proof of a global optimum is implied. Its holdout RMSE is 6.3712 MPa, MAE 4.2896 MPa and $R^2 = 0.8244$.

7.3 Prediction Performance

Table 7.1 gives the approved holdout comparison. Tuned values are mean $\pm$ sample SD over five optimizer seeds. Default SVR is one fixed configuration. Every model is evaluated on

Table 7.1: Holdout results; tuned values are five-seed mean ± sample SD.

Model	RMSE (MPa)	MAE (MPa)	R^2
Default RBF-SVR	9.9069	7.8854	0.5755
Random Search–SVR	6.2328 ± 0.1194	4.3502 ± 0.0253	0.8319 ± 0.0064
DE–SVR	6.2250 ± 0.1104	4.3143 ± 0.0526	0.8324 ± 0.0060

the same 204 holdout observations. DE improves mean RMSE relative to default by approximately 37.2%. Its 0.0078 MPa advantage over Random Search is approximately 0.125% of Random Search RMSE. Both tuned methods therefore provide essentially equivalent holdout performance within this experiment.

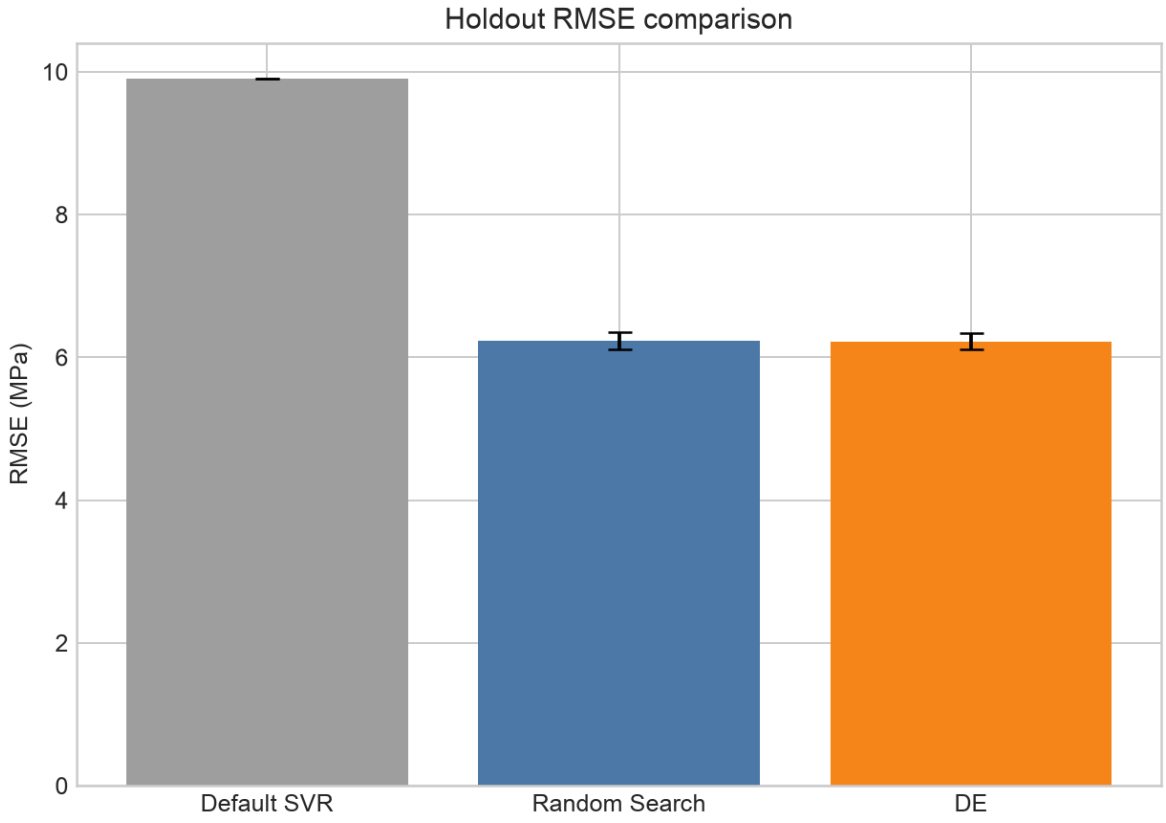


Figure 7.2: Holdout RMSE: default SVR and five-seed tuned-model aggregates. Error bars describe optimizer-seed SD.

7.4 Temporal Performance Analysis

Temporal regime analysis is not applicable to this cross-sectional tabular dataset. Curing age is a predictor rather than a time-ordering rule for samples. No chronological regime or rolling-window result was produced. Age-stratified error analysis could be evaluated in future work under a declared protocol.

7.5 Comparison with Baseline Models

The default model's RMSE of 9.9069 MPa and R^2 of 0.5755 show the limits of its untuned settings on this partition. Random Search reaches $R^2 = 0.8319 \pm 0.0064$ and DE reaches 0.8324 ± 0.0060. The agreement of tuned holdout results is more substantial than their difference. A strong random baseline is necessary to avoid attributing every tuning gain to DE.

7.6 Hyperparameter Importance Analysis

The saved search histories contain candidate values and fitness scores, but the project did not perform an independent hyperparameter-importance experiment. Search correlations are influenced by the sampling strategy and interactions among parameters. They would not establish causal importance. No importance ranking is claimed.

7.7 Feature Importance Results

No permutation-importance, SHAP or ablation study was performed. An RBF-SVR does not supply directly interpretable linear feature coefficients. The report therefore does not invent an ingredient ranking. Any future explanation study should use development data for design decisions and state the uncertainty of its interpretation.

7.8 Error Analysis

Figure 7.3 presents predictions and residuals for the validation-selected demonstration, DE seed 11. All holdout measurements remain in the figure, including large errors. No post-hoc removal was performed. The plot shows the behavior of this selected configuration; it is not an average of predictions from five models. Residual patterns are descriptive. A plot alone does not establish

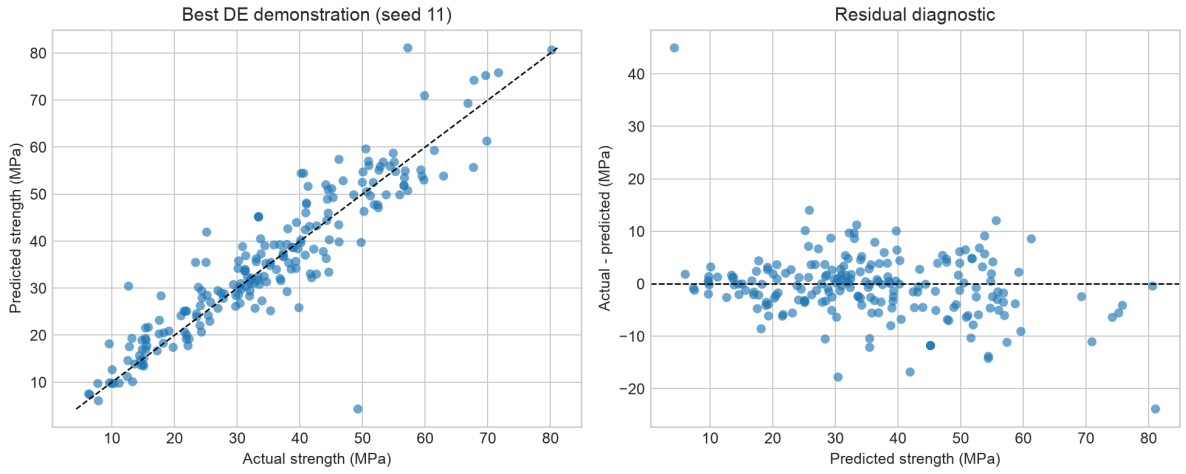


Figure 7.3: Predictions and residuals for the validation-selected demonstration, DE seed 11.

homoscedasticity, normality or absence of bias. The study reports no Breusch–Pagan test or other unsupported inferential diagnostic.

7.9 Computational Performance

The complete search budget is 1,500 candidate evaluations per method, each scored on five development folds. Every attempted candidate is counted. The demonstration's saved final development refit takes 0.4194 seconds in the original experiment environment; this excludes search time and is not an end-to-end runtime claim. The repository retains per-call timing histories for further inspection.

7.10 Robustness Analysis

Table 7.2: All ten predefined validation-selected runs, evaluated on the same holdout.

Method	Seed	RMSE (MPa)	MAE (MPa)	R^2
DE	11	6.3712	4.2896	0.8244
DE	22	6.1675	4.2513	0.8355
DE	33	6.0830	4.3471	0.8400
DE	44	6.2871	4.3860	0.8290
DE	55	6.2163	4.2974	0.8329
Random Search	11	6.3884	4.3422	0.8235
Random Search	22	6.0950	4.3603	0.8393
Random Search	33	6.1456	4.3827	0.8367
Random Search	44	6.3112	4.3520	0.8277
Random Search	55	6.2238	4.3138	0.8325

Across seeds, DE holdout RMSE SD is 0.1104 MPa and Random Search SD is 0.1194 MPa. These values quantify optimization variability on this one holdout. They do not measure robustness to a different dataset, partition, measurement process or concrete source.

The corrected learning curve uses three development-only folds. Within each fold, training subsets are nested samples of complete groups at fractions 0.25, 0.50, 0.75 and 1.00. Parameters remain fixed; there is no retuning or holdout access. Mean validation RMSE declines from 8.3838 to 5.9784 MPa while mean training RMSE rises from 2.7596 to 3.7728 MPa. The remaining gap suggests that the fitted model has lower error on its training subsets than on validation groups.

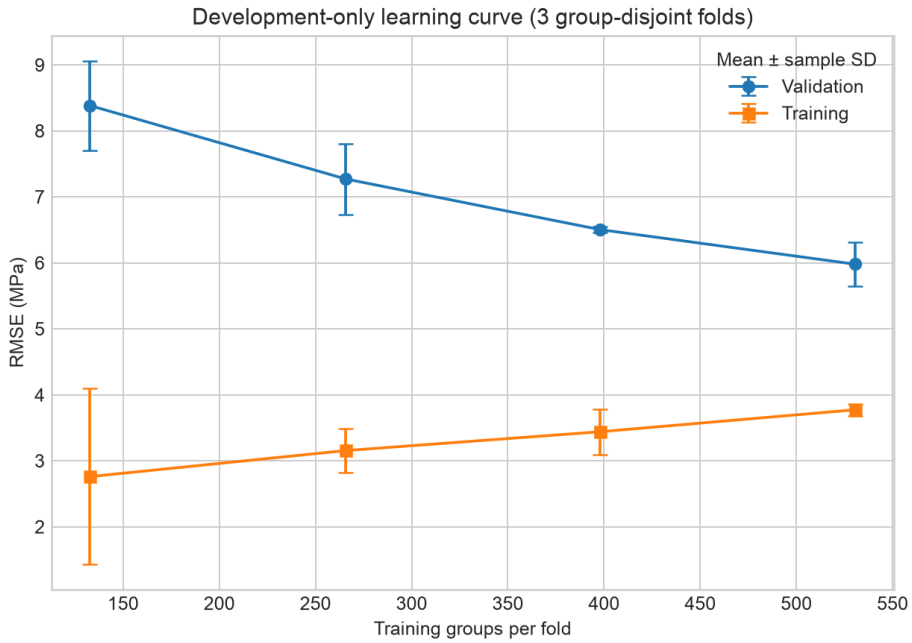


Figure 7.4: Three-fold development diagnostic with nested whole-group samples and fixed demonstration parameters.

These diagnostic values do not replace the five-fold search objective or provide an additional independent generalization estimate.

7.11 Discussion

The most defensible finding is that configuration matters: both tuned methods materially improve default SVR. DE finds slightly lower mean development RMSE, but the advantage almost disappears on holdout. This illustrates why validation optimization progress and independent predictive performance must be reported separately.

The scope remains one dataset, one model, one fixed partition and five optimizer seeds. Historical holdout exposure, finite budgets and environment differences limit interpretation. The evidence supports a reproducible comparison within that scope, rather than a general claim that evolutionary methods outperform random search.

Chapter 8

Comparative Performance Analysis

8.1 Accuracy and Search Budget

Random Search and DE share the same logarithmic bounds, five CV folds, five seed values and 300-call budget. Both tune the same estimator pipeline. The comparison controls opportunity to evaluate candidates, making their accuracy differences more interpretable than comparisons based on unequal search effort.

DE's mean validation RMSE is 5.8797 ± 0.0198 MPa versus 5.9399 ± 0.0287 MPa for Random Search. Holdout means, however, are 6.2250 and 6.2328 MPa respectively. The relative ordering on validation is not sufficient evidence of practically meaningful superiority on holdout.

8.2 Advantages and Limitations

DE offers explicit continuous search operators, a compact implementation and consistent bookkeeping. The group-aware pipeline and complete histories make its decisions auditable. It does not require a derivative of CV error with respect to SVR parameters.

Its limitations include many repeated model fits, dependence on parameter bounds and fixed control settings, and a finite search that cannot certify global optimality. Random Search is simpler and achieves almost the same holdout accuracy in this case. The project does not measure deployment latency, certify structural mixtures, or demonstrate transfer to external laboratory data.

Chapter 9

Applications and Case Studies

9.1 Applications

Within a validated domain, a strength-prediction model can support exploratory comparison of mixture measurements and provide an additional numerical estimate for research. Ingredient quantities and curing age must use the same definitions and units as training. Such estimates should be reviewed alongside physical test results and domain knowledge.

The current model is a research artifact. The project does not demonstrate field deployment, laboratory qualification, mixture optimization or a reduction in cement consumption. Predicting strength from supplied inputs is also different from searching for a mixture that satisfies engineering constraints.

9.2 Case Study: UCI Concrete Compressive Strength

The implemented case study uses a single public dataset and a fixed group-disjoint holdout. The demonstration is reproducible from its recorded parameters and dataset partition. Its error should be interpreted at the dataset level and not as a tolerance guarantee for an individual construction decision.

A practical extension would collect external measurements from an identified source and evaluate a frozen model without retuning on that external test set. Until such evidence exists, application claims remain prospective.

Chapter 10

Conclusion

10.1 Key Findings

This project implemented and evaluated DE/rand/1/bin for RBF-SVR hyperparameter optimization on concrete compressive strength. A fixed group-disjoint protocol, fold-local scaling and equal initialization-inclusive search budgets allowed a controlled comparison with default SVR and Random Search.

DE achieves mean holdout RMSE of 6.2250 ± 0.1104 MPa, MAE of 4.3143 ± 0.0526 MPa and R^2 of 0.8324 ± 0.0060. The mean RMSE reduction from default SVR is approximately 37.2%. Random Search achieves essentially the same holdout accuracy. The experiment therefore demonstrates the value of tuning and an auditable evolutionary implementation, while providing no basis for claiming significant general superiority of DE.

10.2 Key Learnings

Data isolation and honest experiment accounting are central to a useful optimizer comparison. Repeated input tuples need group-aware splits; scaling must be fitted within folds; every candidate evaluation must count toward the budget. The demonstration model and five-seed aggregate answer different questions and must remain distinct.

Preserving original provenance, complete histories and fixed selections made it possible to audit results across environments. This reproducibility is a stronger project outcome than an unsupported accuracy claim based on a favorable seed.

Chapter 11

Future Work

11.1 Methodological Enhancements

Future experiments could compare adaptive DE controls or other search methods using a protocol fixed before evaluation. Population size, mutation weight and crossover rate should be varied systematically with equal budgets. Such experiments must be presented as new work rather than retroactively improving the current frozen result.

11.2 Data and Validation

Repeated outer group splits or nested group cross-validation would assess sensitivity to partition choice. External concrete datasets would test transfer beyond the current measurement collection. Age-stratified errors, uncertainty intervals and ingredient-level interpretation could add useful context if designed and validated carefully.

Because the current holdout has already been evaluated, it should not become a feedback channel for repeated improvement. New model-selection decisions require a clearly separated evaluation design. Feature importance and ablation studies should report their methods and uncertainty rather than infer rankings from a single selected configuration.

11.3 Sustainable Development Goals

Reliable materials modeling could be relevant to SDG 9 (Industry, Innovation and Infrastructure) and SDG 12 (Responsible Consumption and Production). This is a potential connection, not a measured sustainability outcome. The present experiment does not quantify material savings, carbon reduction or safety improvements. A future constrained mixture-design study would need environmental data and engineering validation to assess those outcomes.

References

[1] I.-C. Yeh, *Concrete compressive strength*, Dataset, UCI Machine Learning Repository, 1998. DOI: 10.24432/C5PK67 [Online]. Available: https://archive.ics.uci.edu/dataset/165/concrete+compressive+strength

[2] I.-C. Yeh, "Modeling of strength of high-performance concrete using artificial neural networks," *Cement and Concrete Research*, vol. 28, no. 12, pp. 1797–1808, 1998. DOI: 10.1016/S0008-8846(98)00165-3

[3] scikit-learn developers. "Svr: Epsilon-support vector regression. "[Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVR.html

[4] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012. [Online]. Available: https://www.jmlr.org/papers/v13/bergstra12a.html

[5] R. Storn and K. Price, "Differential evolution: A simple and efficient heuristic for global optimization over continuous spaces," *Journal of Global Optimization*, vol. 11, pp. 341–359, 1997. DOI: 10.1023/A:1008202821328

[6] A. J. Smola and B. Schölkopf, "A tutorial on support vector regression," *Statistics and Computing*, vol. 14, pp. 199–222, 2004. DOI: 10.1023/B:STCO.0000035301.49549.88

[7] J. H. Holland, *Adaptation in Natural and Artificial Systems, An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. MIT Press, 1992, Original work published 1975. [Online]. Available: https://mitpress.mit.edu/9780262581110/adaptation-in-natural-and-artificial-systems/

[8] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proceedings of ICNN'95, International Conference on Neural Networks*, vol. 4, 1995, pp. 1942–1948. DOI: 10.1109/ICNN.1995.488968

[9] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant system: Optimization by a colony of cooperating agents," *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, vol. 26, no. 1, pp. 29–41, 1996. DOI: 10.1109/3477.484436

[10] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983. DOI: 10.1126/science.220.4598.671

GitHub Repo Link

https://github.com/R-V-Abhishek/concrete-strength-de-svr